Dynamic Road Junction Graph with Insertion, Deletion, and Neighbour Queries

Problem Statement 2: Modified Program

1. Objective

A menu-driven C++ program manages a road network that changes over time. The network is stored as an undirected adjacency list, and the user can insert new junctions with their roads, remove existing roads, query degrees and direct neighbours, and display the updated graph. Changes are applied to the list immediately, and the program runs until the user chooses Exit.

2. Menu Operations

- **1. Display current graph:** prints the adjacency list.
- **2. Insert a new junction:** adds a vertex with id = (current count + 1) and connects it to the chosen existing junctions.
- **3. Remove a road:** deletes the edge between two junctions from both adjacency lists.
- **4. Degree of a junction:** the number of roads at that junction.
- **5. Direct neighbours:** the junctions directly connected to the selected junction.
- **6. Updated graph summary:** total junctions, total roads, a degree table, and the adjacency list.
- **7. Exit.**

3. Design Notes

- **Adjacency list:** a vector of vectors. Index 0 is unused so junctions are numbered 1 to n directly.
- **Dynamic growth:** inserting a junction appends a new list at the end, so no existing ids change.
- **Deletion:** the road is removed from both lists, and the road counter is decremented.
- **Validation:** invalid junction numbers, self-loops, and duplicate roads are rejected. Non-integer input is re-prompted instead of crashing or looping.
- **Complexity:** neighbour and duplicate checks are O(deg(v)); insertion of a junction with k targets is O(k * deg); removal is O(deg(u) + deg(v)); degree lookup is O(1).

4. Source Code (dynamic_road_network.cpp)

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <limits>
using namespace std;

class DynamicRoadNetwork {
    vector<vector<int>> adj;   // adjacency list; index 0 unused, junctions are 1..n
    int roads = 0;             // total number of undirected edges

    bool isJunction(int v) const { return v >= 1 && v < static_cast<int>(adj.size()); }
    bool hasRoad(int u, int v) const {
        return find(adj[u].begin(), adj[u].end(), v) != adj[u].end();
    }

public:
    explicit DynamicRoadNetwork(int n) : adj(n + 1) {}

    int junctionCount() const { return static_cast<int>(adj.size()) - 1; }
    int roadCount() const { return roads; }

    // Adds a road between existing junctions u and v (used while building the graph)
    bool addRoad(int u, int v) {
        if (!isJunction(u) || !isJunction(v) || u == v || hasRoad(u, v)) return false;
        adj[u].push_back(v);
        adj[v].push_back(u);
        roads++;
        return true;
    }

    // Inserts a new junction connected to the given existing junctions
    int insertJunction(const vector<int>& targets, int& rejected) {
        adj.emplace_back();                       // new vertex gets id = old count + 1
        int id = junctionCount();
        rejected = 0;
        for (int t : targets) {
            if (!addRoad(id, t)) rejected++;      // invalid, self, or duplicate targets are skipped
        }
        return id;
    }

    // Removes the road between u and v, if it exists
    bool removeRoad(int u, int v) {
        if (!isJunction(u) || !isJunction(v) || !hasRoad(u, v)) return false;
        adj[u].erase(find(adj[u].begin(), adj[u].end(), v));
        adj[v].erase(find(adj[v].begin(), adj[v].end(), u));
        roads--;
        return true;
    }

    int degree(int v) const { return static_cast<int>(adj[v].size()); }

    const vector<int>& neighbours(int v) const { return adj[v]; }

    void displayGraph() const {
        cout << "\nAdjacency List:\n";
        for (int i = 1; i <= junctionCount(); i++) {
            cout << "Junction " << setw(2) << i << " -> ";
            if (adj[i].empty()) {
                cout << "(no roads)";
            } else {
                for (size_t j = 0; j < adj[i].size(); j++) {
                    cout << adj[i][j];
                    if (j + 1 < adj[i].size()) cout << ", ";
                }
            }
            cout << "\n";
        }
    }
    void displaySummary() const {
```

```
        cout << "\nUpdated Graph Summary\n";
        cout << "Total junctions : " << junctionCount() << "\n";
        cout << "Total roads     : " << roads << "\n\n";
        cout << setw(10) << "Junction" << setw(10) << "Degree" << "\n";
        cout << string(20, '-') << "\n";
        for (int i = 1; i <= junctionCount(); i++)
            cout << setw(10) << i << setw(10) << degree(i) << "\n";
        displayGraph();
    }
};

// Reads an integer safely; re-prompts on bad input instead of looping forever
int readInt(const string& prompt) {
    int x;
    while (true) {
        cout << prompt;
        if (cin >> x) return x;
        cout << "Invalid input. Please enter an integer.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int main() {
    int n = readInt("Enter the number of junctions: ");
    if (n <= 0) { cout << "Number of junctions must be positive.\n"; return 1; }
    int m = readInt("Enter the number of roads: ");
    if (m < 0) { cout << "Number of roads cannot be negative.\n"; return 1; }

    DynamicRoadNetwork net(n);
    cout << "Enter each road as two junction numbers (u v):\n";
    for (int i = 0; i < m; i++) {
        int u = readInt("Road " + to_string(i + 1) + ": ");
        int v = readInt("   to: ");
        if (!net.addRoad(u, v))
            cout << "  Invalid or duplicate road ignored.\n";
    }
    cout << "\nNetwork created with " << net.junctionCount() << " junctions and "
         << net.roadCount() << " roads.\n";

    while (true) {
        cout << "\n========== ROAD NETWORK MENU ==========\n"
             << "1. Display current graph\n"
             << "2. Insert a new junction with connections\n"
             << "3. Remove a road between two junctions\n"
             << "4. Calculate degree of a junction\n"
             << "5. Display direct neighbours of a junction\n"
             << "6. Display updated graph (summary)\n"
             << "7. Exit\n";
        int choice = readInt("Enter your choice: ");

        switch (choice) {
        case 1:
            net.displayGraph();
            break;

        case 2: {
            int k = readInt("How many existing junctions will the new junction connect to? ");
            if (k < 0) { cout << "Count cannot be negative.\n"; break; }
            vector<int> targets;
            for (int i = 0; i < k; i++)
                targets.push_back(readInt("  Connect to junction " + to_string(i + 1) + ": "));
            int rejected = 0;
            int id = net.insertJunction(targets, rejected);
            cout << "New junction " << id << " added with " << (k - rejected)
                 << " road(s).\n";
            if (rejected > 0)
                cout << rejected << " connection(s) skipped (invalid, self, or duplicate).\n";
            break;
        }

        case 3: {
            int u = readInt("Enter first junction: ");
            int v = readInt("Enter second junction: ");
```

```
            if (net.removeRoad(u, v))
                cout << "Road between " << u << " and " << v << " removed.\n";
            else
                cout << "No such road exists between " << u << " and " << v << ".\n";
            break;
        }

        case 4: {
            int v = readInt("Enter junction number: ");
            if (!(v >= 1 && v <= net.junctionCount())) { cout << "Invalid junction.\n"; break; }
            cout << "Degree of junction " << v << " is " << net.degree(v) << ".\n";
            break;
        }

        case 5: {
            int v = readInt("Enter junction number: ");
            if (!(v >= 1 && v <= net.junctionCount())) { cout << "Invalid junction.\n"; break; }
            cout << "Direct neighbours of junction " << v << ": ";
            if (net.neighbours(v).empty()) {
                cout << "none\n";
            } else {
                for (int nb : net.neighbours(v)) cout << nb << " ";
                cout << "\n";
            }
            break;
        }

        case 6:
            net.displaySummary();
            break;

        case 7:
            cout << "Exiting road network manager. Goodbye!\n";
            return 0;

        default:
            cout << "Invalid choice. Please select 1 to 7.\n";
        }
    }
}
```

5. Compilation and Execution

```
g++ -std=c++17 -Wall -Wextra -o dynamic_road_network dynamic_road_network.cpp
./dynamic_road_network
```

6. Sample Session

Initial network: 5 junctions, roads 1-2, 1-3, 2-3, 4-5. The session below inserts junction 6, removes road 1-2, and then runs queries. Repeated menu headers are omitted for brevity.

```
Enter the number of junctions: 5
Enter the number of roads: 4
Road 1:    to: 1 2
Road 2:    to: 1 3
Road 3:    to: 2 3
Road 4:    to: 4 5

Network created with 5 junctions and 4 roads.

Enter your choice: 1        (Display current graph)
Adjacency List:
Junction  1 -> 2, 3
Junction  2 -> 1, 3
Junction  3 -> 1, 2
Junction  4 -> 5
Junction  5 -> 4

Enter your choice: 2        (Insert a new junction)
How many existing junctions will the new junction connect to? 2
  Connect to junction 1: 1
```

```
  Connect to junction 2: 2
New junction 6 added with 2 road(s).

Enter your choice: 3        (Remove a road)
Enter first junction: 1
Enter second junction: 2
Road between 1 and 2 removed.

Enter your choice: 3        (Remove a road that does not exist)
Enter first junction: 4
Enter second junction: 3
No such road exists between 4 and 3.

Enter your choice: 1
Adjacency List:
Junction  1 -> 3, 6
Junction  2 -> 3
Junction  3 -> 1, 2
Junction  4 -> 5, 6
Junction  5 -> 4
Junction  6 -> 1, 4

Enter your choice: 5        (Direct neighbours)
Enter junction number: 2
Direct neighbours of junction 2: 3

Enter your choice: 4        (Degree)
Enter junction number: 5
Degree of junction 5 is 1.

Enter your choice: 7
Exiting road network manager. Goodbye!
```

7. Observations

After junction 6 is inserted with roads to junctions 1 and 2, and road 1-2 is removed, junction 1 has neighbours 3 and 6, and junction 2 has only neighbour 3. Attempting to remove a non-existent road reports a message and leaves the graph unchanged.